

# Hunting For Process Injection & Proactive API Monitoring

## LAB 8

### Scenario

Attackers love hiding/injecting malicious code into processes. In this lab, you will learn how to hunt for various process injection techniques and how to leverage userland API monitoring for more effective hunts.

### Goals

The learning objective of this lab is to practice detecting

### What you will learn

You will learn an alternative way of detecting process injection and how proactive API monitoring can result in better hunting for process injection.

### Recommended tools

- <https://github.com/marcosd4h/memhunter>
- <https://github.com/y3n11/Captain>

All the tools are present in the **C:\Users\Administrator\Desktop\Tools** folder

# Tasks

## Task 1. Practice detecting process injection using memhunter, minjector & Process Hacker 2

According to the *Memhunter* developer: "The detection process is performed through a combination of endpoint data collection and memory inspection scanners. The tool is a standalone binary that, upon execution, deploys itself as a windows service. Once running as a service, *memhunter* starts the collection of ETW events that might indicate code injection attacks. The live stream of collected data events is feed into memory inspection scanners that use detection heuristics to down select the potential attacks. The entire detection process does not require human intervention, neither memory dumps, and it can be performed by the tool itself at scale."

*memhunter* and *minjector* reside at *C:\Users\Administrator\Desktop\Tools\memhunter*

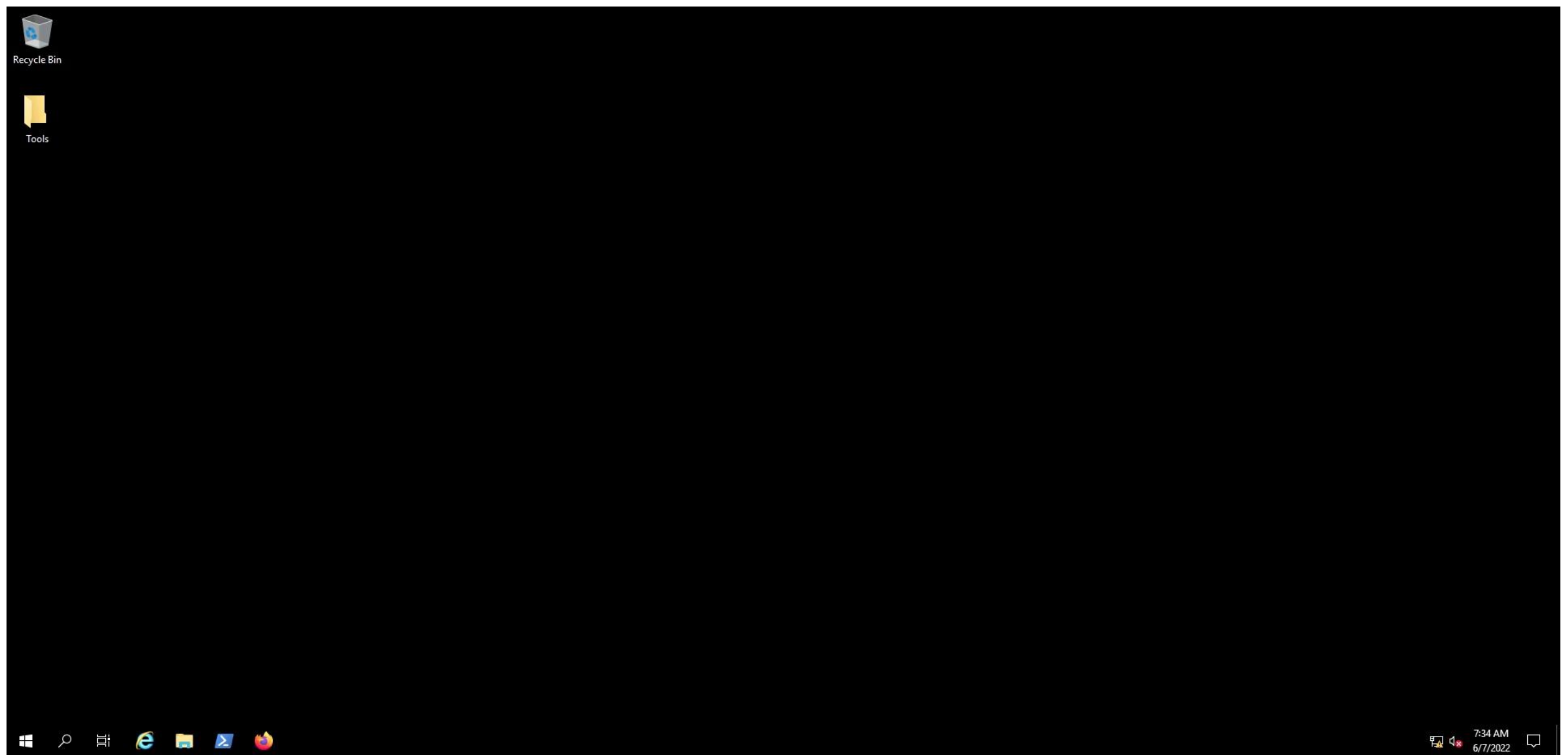
Perform a process injection through *minjector.exe* (use option 1) against *notepad.exe* and detect the attack through *memhunter.exe* and *Process Hacker 2* (available on the Desktop)

## Task 2. Practice detecting process injection through proactive API monitoring

According to the *Captain* developer:"Captain is an endpoint monitoring tool that aims at spotting malicious events through API hooking, improving the process of threat hunting analysis . When a new process is created, Captain will inject a dll into it hooking some Windows API functions."

1. Close the *notepad.exe* process where you injected a DLL during Task 1.
2. **As an administrator**, start the monitoring PowerShell script (*Monitor.ps1*) of the Captain detection tool. The script resides at: *C:\Users\Administrator\Desktop\Tools\Captain*
3. Open a new *notepad.exe* process and perform a process injection through *minjector.exe*, using option 5 this time (Reflective DLL Injection)
4. Navigate to *C:\ProgramData\Captain\Reporting* and read the content of *events.json*.

# SOLUTIONS



Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

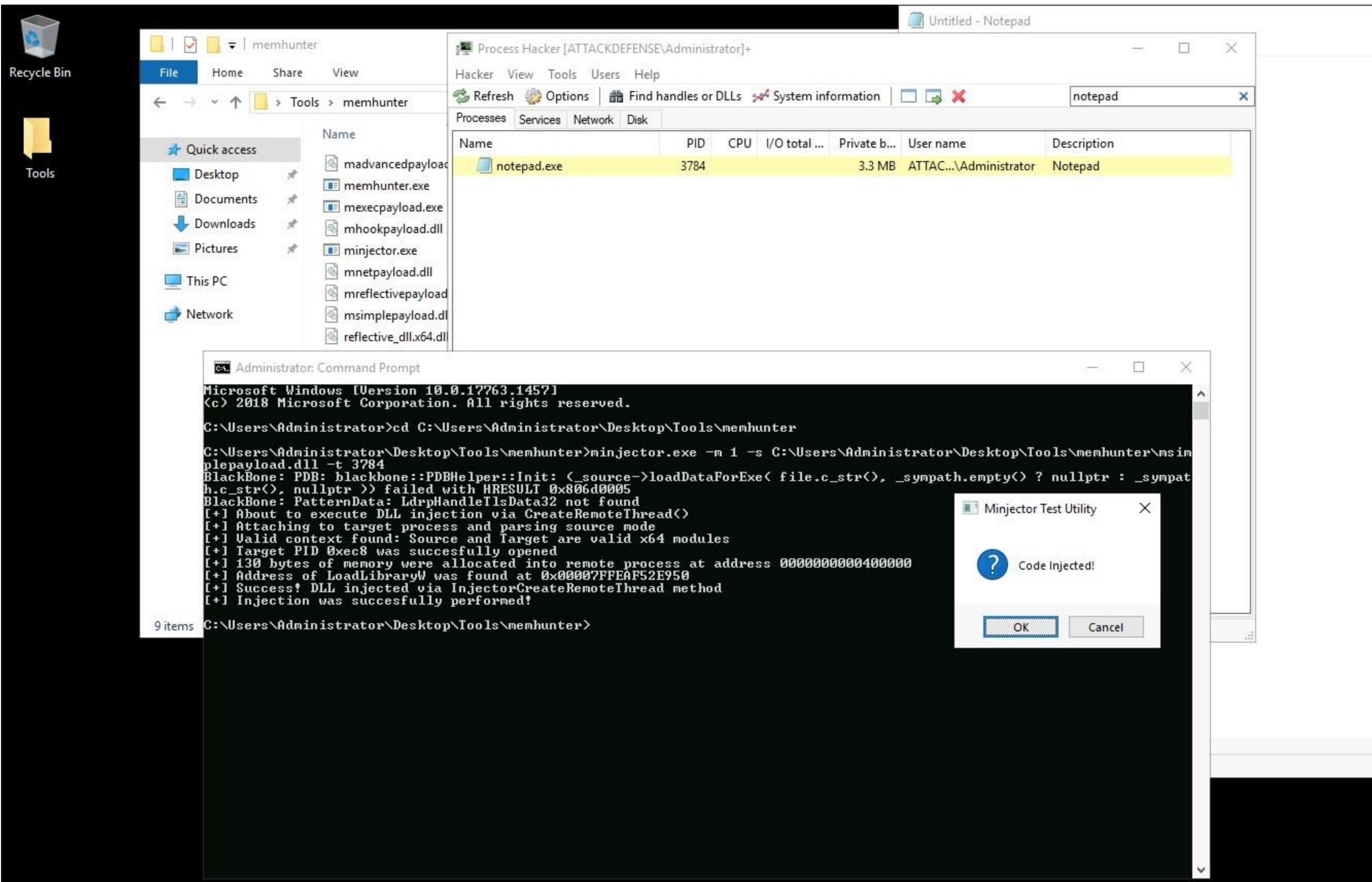
## Task 1. Practice detecting process injection using memhunter, minjector & Process Hacker 2

To perform a process injection against notepad.exe, perform the following.

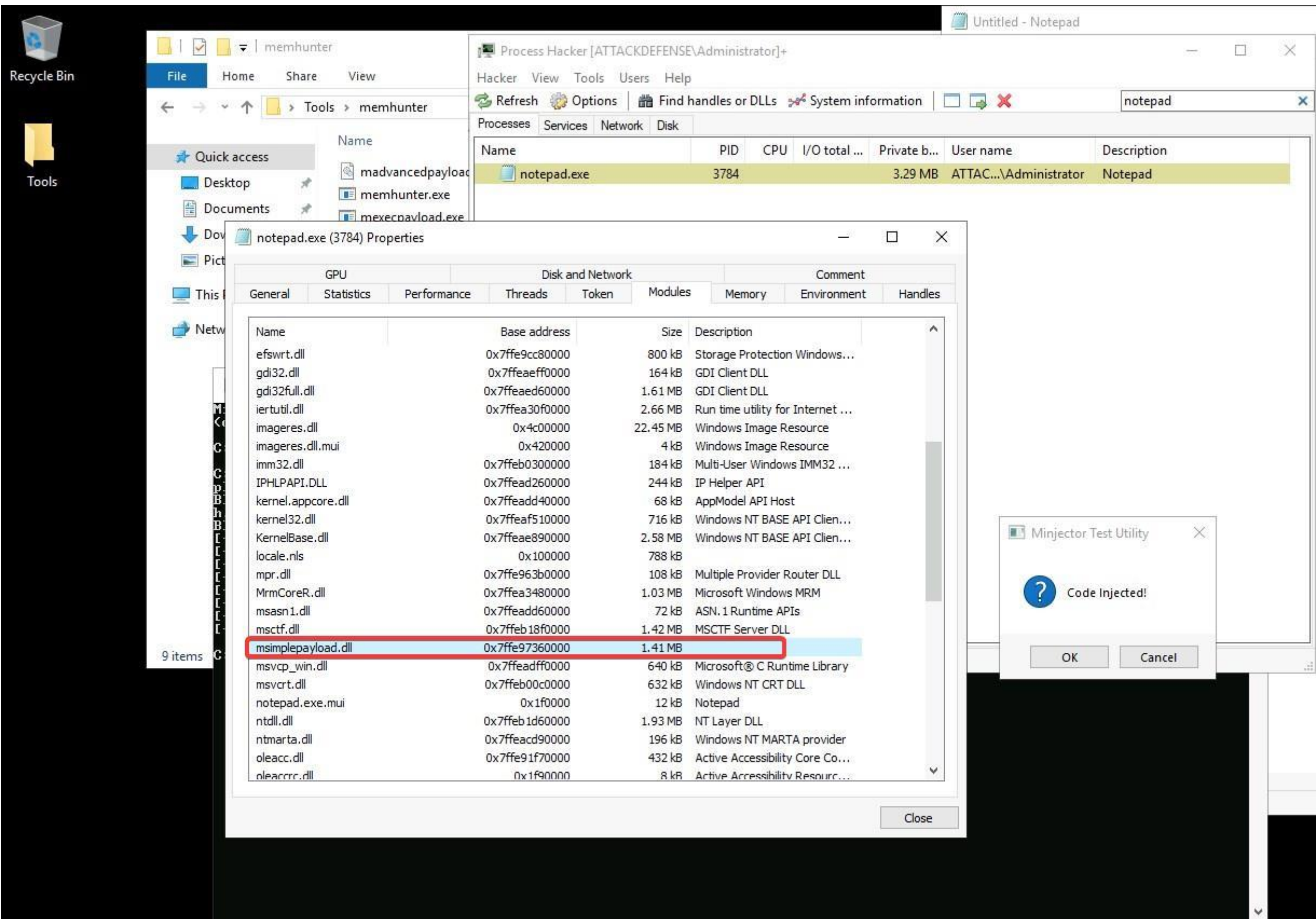
- Run *Process Hacker 2* as an administrator.
- Run notepad.exe and identify its PID through Process Hacker 2

- Open cmd.exe as **an administrator** and execute:

```
cd C:\Users\Administrator\Desktop\Tools\memhunter  
minjector.exe -m 1 -s C:\Users\Administrator\Desktop\Tools\memhunter\msimplepayload.dll -t PID_of_notepad_exe
```



By double-clicking the notepad.exe process inside *Process Hacker 2* and browsing the "*Modules*" tab, we can see the injected dll.



We could have detected the process injection activity through *memhunter.exe*, as follows.

**Don't close the notepad.exe process where you injected the DLL previously!!!**

Open cmd.exe as **an administrator** and execute:

```
cd C:\Users\Administrator\Desktop\Tools\memhunter
memhunter.exe -r
```

**Administrator: Command Prompt**

```
[+] Injection was successfully performed!
C:\Users\Administrator\Desktop\Tools\memhunter>memhunter.exe -r
BlackBone: PDB: blackbone::PDBHelper::Init: (<source->loadDataForExe< file.c_str(), _sympath.empty() ? nullptr : _sympat
h.c_str(), nullptr >>) failed with HRESULT 0x806d0005
BlackBone: PatternData: LdrpHandleTlsData32 not found

===== New Suspicious Process Found: C:\Windows\System32\notepad.exe =====
Hunter Module: SuspiciousModules
Process ID: 3784
Process Name: notepad.exe
Process Path: c:\windows\system32\notepad.exe
Process Command Line: "C:\Windows\system32\notepad.exe"
Process Mr Threads: 6
Process Base Priority: 32
Suspicious Module Name: SuspiciousModules
Suspicious Module Path: memory
Suspicious Thread Base Addr: 0x00007FFE97736000

===== New Suspicious Process Found: C:\Windows\System32\notepad.exe =====
Hunter Module: SuspiciousCallStack
Process ID: 3784
Process Name: notepad.exe
Process Path: c:\windows\system32\notepad.exe
Process Command Line: "C:\Windows\system32\notepad.exe"
Process Mr Threads: 6
Process Base Priority: 32
Suspicious Module Name: SuspiciousCallStack
Suspicious Module Path: memory
Suspicious Thread ID: 1624
Suspicious Thread Priority: 0
Dumped Suspicious Callstack:
  stackFrameAddress: 140731829785124
  symbolAddress: 140731829785104
  baseOfImageAddress: 140731829780480
  loadedImageName: C:\Windows\System32\win32u.dll
  imageName: C:\Windows\System32\win32u.dll
  moduleName: win32u
  symbolName: NtUserWaitMessage

  stackFrameAddress: 140731838230008
  symbolAddress: 140731838229056
  baseOfImageAddress: 140731838038016
  loadedImageName: C:\Windows\System32\USER32.dll
  imageName: C:\Windows\System32\USER32.dll
  moduleName: USER32
  symbolName: DialogBoxIndirectParamAorW

  stackFrameAddress: 140731838229460
  symbolAddress: 140731838229056
  baseOfImageAddress: 140731838038016
  loadedImageName: C:\Windows\System32\USER32.dll
  imageName: C:\Windows\System32\USER32.dll
  moduleName: USER32
  symbolName: DialogBoxIndirectParamAorW

  stackFrameAddress: 140731838510902
  symbolAddress: 140731838508880
  baseOfImageAddress: 140731838038016
  loadedImageName: C:\Windows\System32\USER32.dll
  imageName: C:\Windows\System32\USER32.dll
  moduleName: USER32
  symbolName: SoftModalMessageBox

  stackFrameAddress: 140731838505333
  symbolAddress: 140731838498096
  baseOfImageAddress: 140731838038016
```

**Process Hacker [ATTACKDEFENSE\Administrator]+**

Hacker View Tools Users Help

Refresh Options Find handles or DLLs System information notepad

Processes Services Network Disk

| Name        | PID  | CPU  | I/O total ... | Private b... | User name             | Description |
|-------------|------|------|---------------|--------------|-----------------------|-------------|
| notepad.exe | 3784 | 0.08 |               | 3.11 MB      | ATTAC...Administrator | Notepad     |

CPU Usage: 2.65% Physical memory: 1.19 GB (14.93%) Processes: 103

Windows (CRLF) Ln 1, Col 1

7:46 AM 6/7/2022

Wait for 2-3 minutes.

*Memhunter* successfully identified the process injection activity.

Try to detect process injection through the Reflective DLL injection and Process Hollowing options of minjector.exe with both *Memhunter* and *Process Hacker 2*.

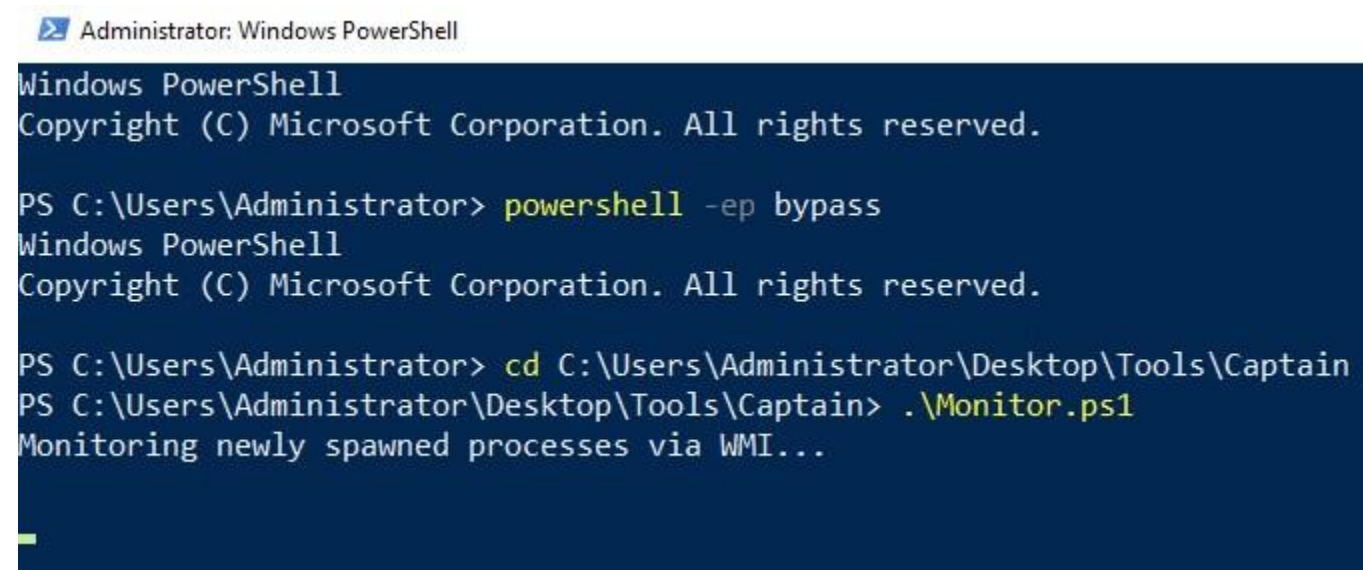
- Regarding Reflective DLL injection, focus on the "*Threads*" tab of *Process Hacker 2*
- Regarding Process Hollowing, focus on the "*General*" and "*Modules*" tabs of *Process Hacker 2*

## Task 2. Hunt for suspicious network connections

Let's first start Captain's monitoring script, as follows.

Open PowerShell **as an administrator**, and execute.

```
powershell -ep bypass
cd C:\Users\Administrator\Desktop\Tools\Captain
.\Monitor.ps1
```



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Tools\Captain
PS C:\Users\Administrator\Desktop\Tools\Captain> .\Monitor.ps1
Monitoring newly spawned processes via WMI...
```

Now, let's run a new notepad.exe process. Once we do that, we should see the below inside Captain's monitoring script.

Administrator: Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> powershell -ep bypass
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> cd C:\Users\Administrator\Desktop\Tools\Captain
PS C:\Users\Administrator\Desktop\Tools\Captain> .\Monitor.ps1
Monitoring newly spawned processes via WMI...

(1) New process spawned:
PID:          4904
Name:         notepad.exe
PPID:         4776
Parent name:   explorer
CommandLine:  "C:\Windows\system32\notepad.exe"
Trying to suspend process: 4904
Process is suspended.
Dll Injected !!
Trying to resume process: 4904
```

It is time we perform our Reflective DLL injection against *notepad.exe*, using minjector.exe.

Open cmd.exe as **an administrator** and execute:

```
cd C:\Users\Administrator\Desktop\Tools\memhunter
minjector.exe -m 5 -s C:\Users\Administrator\Desktop\Tools\memhunter\reflective_dll.x64.dll -t PID_of_notepad_exe
```

Administrator: Windows PowerShell

Parent name: cmd  
CommandLine: \\?\C:\Windows\system32\conhost.exe 0x4  
Process ignored as per configuration.

(4) New process spawned:

PID: 4236  
Name: cmd.exe  
PPID: 4776  
Parent name: explorer  
CommandLine: "C:\Windows\system32\cmd.exe"  
Trying to suspend process: 4236  
Process is suspended.  
Dll Injected !!  
Trying to resume process: 4236

(5) New process spawned:

PID: 1904  
Name: backgroundTaskHost.exe  
PPID: 928  
Parent name: svchost  
CommandLine: "C:\Windows\system32\backgroundTaskHost.exe" -ServerName:  
.mca  
Trying to suspend process: 1904  
Process is suspended.  
Dll Injected !!  
Trying to resume process: 1904

(6) New process spawned:

PID: 1972  
Name: ProcessHacker.exe  
PPID: 4776  
Parent name: explorer  
CommandLine: "C:\Program Files\Process Hacker 2\ProcessHacker.exe"  
Trying to suspend process: 1972  
Process is suspended.  
Dll Injected !!  
Trying to resume process: 1972

Process Hacker [ATTACKDEFENSE\Administrator]+

Hacker View Tools Users Help

Refresh Options Find handles or DLLs System information

Processes Services Network Disk

| Name        | PID  | CPU | I/O total ... | Private b... | User name |
|-------------|------|-----|---------------|--------------|-----------|
| notepad.exe | 4904 |     |               | 2.88 MB      | ATTAC...A |

Administrator: Command Prompt

Microsoft Windows [Version 10.0.17763.1457]  
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>C:\Users\Administrator\Desktop\Tools\memhunter\minjector.exe -m 5 -s C:\Users\A  
p\Tools\memhunter\reflective\_dll.x64.dll -t 4904  
BlackBone: PDB: backbone::PDBHelper::Init: (<\_source->loadDataForExe( file.c\_str(), \_sympath.empty() ?  
h.c\_str(), nullptr )) failed with HRESULT 0x806d0005  
BlackBone: PatternData: LdrpHandleTlsData32 not found  
[+] About to execute DLL injection via Reflective DLL injection  
[+] Valid context found: Source and Target are valid x64 modules  
[+] Target PID 0x1328 was successfully opened  
[+] About to allocate 66560 bytes of memory for memory DLL  
[+] Success! DLL injected via InjectorReflectiveDLL method  
[+] Injection was successfully performed!

C:\Users\Administrator>

Reflective DLL Injection X

Hello from DIIMain!

OK

The reflective DLL injection attempt should be successful and a message box mentioning "Hello from DllMain" should be visible (click on the notepad icon on the taskbar if it is not visible).

Finally navigate to *C:\ProgramData\Captain\Reporting* and read the content of *events.json*. You should notice functions that are related with process injection being called and used against *notepad.exe*.

```
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:51:39:477",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:2:39",
  "function": {
    "arguments": {
      "lpLibFileName": "kernel32"
    },
    "func_name": "LoadLibraryW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:19:933",
  "function": {
    "arguments": {
      "lpApplicationName": "C:\\Users\\Administrator\\Desktop\\Tools\\memhunter\\minjector.exe",
      "lpCommandLine": "C:\\Users\\Adminis
      -m 5 -s C:\\Users\\Administrator\\Desktop\\Tools\\memhunter\\reflective_dll.x64.dll -t 4904"
    },
    "func_name": "CreateProcessW",
    "proc_id": "4236",
    "proc_name": "C:\\Windows\\system32\\cmd.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:20:11",
  "function": {
    "arguments": {
      "lpLibFileName": "USER32.dll"
    },
    "func_name": "LoadLibraryA",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
},
{
  "event_time": "6/7/2022 7:53:20:11",
  "function": {
    "arguments": {
      "lpLibFileName": "KERNEL32.dll"
    },
    "func_name": "LoadLibraryA",
    "proc_id": "4904",
    "proc_name": "C:\\Windows\\system32\\notepad.exe"
  }
}
```

Feel free to explore all functionality of the Captain tool. It will give you a nice idea of how actual EDR work.

<https://github.com/y3n11/Captain>